

Module 6

SPI — UVM+SV Verification

Learning objectives

- SPI UVM agent: Transaction (SpiTransaction), sequence (SpiSequence), driver (SpiDriver), monitor (S...
- Interface: spi_master_if (clk, rst_n, clk_div_tick, start, data_in, sclk, mosi, cs_n, busy, done) c...
- Mode 0 monitoring: SCLK idle low; capture on rising edge; monitor samples MOSI on each rising SCLK,...

Prerequisites

- Module 1 (spec → RTL), Module 2 (UVM+SV, uvm_smoke), Module 5 (SPI protocol + RTL + basic TB).
- Verilator, Make, C++ compiler, UVM (UVM_HOME or vendored UVM).

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["SPI protocol recap (Mode 0)"]

T2["Toolchain"]

T1 --> T2

T2 --> N["I²C"]

Extend SPI verification to ****UVM+SV**** — SPI agent (transaction, sequence, driver, monitor, scoreboard); run on Verilator.

Overview

- Module 6 builds on Module 5 (SPI protocol + RTL + basic testbench):

How to learn this module

Suggested learning path

- Read the detailed SPI learning guide:
SPI_LEARNING_GUIDE.md — what SPI is, how it works
(signals, M...
- Read the protocols + UVM overview:
LEARNING_GUIDE_PROTOCOLS_AND_UVM.md — Part B § 5.
How UVM Maps t...

Design architecture

RTL / block architecture

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

subgraph dut["SPI DUT (Module 5)"]

DIV["clk_div"]

M["spi_master"]

DIV --> M

end

Module 6: DUT / block hierarchy (see module6/examples/).

Verification environment architecture

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart TB

SEQ["SpiSequence"] --> DRV["SpiDriver\nstart, data_in"]

DRV --> DUT["spi_master"]

DUT --> WIRE["sclk, mosi, cs_n"]

WIRE --> MON["SpiMonitor\nsample MOSI @ rising SCLK"]

MON --> SB["SpiScoreboard\nMSB first, Mode 0"]

Module 6: UVM layers, agents, and checkers for this phase.

1. SPI DUT (reused from Module 5)

- spi_master + clk_div in
 `module6/examples/spi\_uvm/dut/`.
- spi_master_if connects UVM to DUT pins.

2. UVM agent structure

- SpiTransaction = one byte; SpiSequence = directed list; SpiDriver waits on `done`.
- SpiMonitor samples MOSI on rising SCLK while `cs\_n` is low (Mode 0, MSB first).
- SpiScoreboard compares expected vs `observed\_mosi`.

3. Bus vs DUT checking

- Monitor validates wire behavior; `done` alone is not sufficient for sign-off.

Verification & testing methods

Testing and closure flow

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

SEQ["SpiSequence"] --> DRV["SpiDriver"]

DRV --> DUT["spi_master"]

DUT --> MON["Monitor SCLK/MOSI"]

MON --> SB["Scoreboard"]

Module 6: how tests, checks, and metrics close verification goals.

1. Mode 0 in the testbench

- Monitor must use rising-edge capture and falling-edge change — matches CPOL/CPHA=0.

2. Directed regression

- Same byte pattern as UART UVM (0x00 ... 0xFF) for apples-to-apples learning.

3. Extensions

- MISO reads, random data, mode parameters — see
[LEARNING_GUIDE §
7](LEARNING_GUIDE_PROTOCOLS_AND_UV...

Syllabus topics

1. SPI protocol recap (Mode 0)

- `cs\_n` frames; 8 SCLK cycles per byte; MSB first on MOSI.

2. Toolchain

- ``make SIM=verilator TEST=test_spi_uvm`;`
 ``+UVM_TESTNAME=SpiTest`.`

Hands-on examples

Module 6 self-check

```
$ ./scripts/module6.sh --help
```

Terminal: module6_check

```
[[0;36m===== [[0m
[[0;36mModule 6: Demo commands (SPI UVM example) [[0m
[[0;36m===== [[0m

From repo root:

# 1) Environment sanity:
verilator --version
make --version
echo "$UVM_HOME"

# 2) Run the SPI UVM example (inside the example directory):
cd module6/examples/spi_uvm
make SIM=verilator TEST=test_spi_uvm

# 3) Inspect logs and waveforms:
ls obj_dir
# If a VCD is produced:
ls *.vcd

[[1;33m[INFO] [[0m See module6/EXAMPLES.md and module6/examples/spi_uvm/README.md for more details.
```

Exercise scaffold

```
./scripts/module6.sh --scaffold
```


Example 1: SPI UVM

- UVM phases and reporting (DRIVER, MONITOR, SCOREBOARD).
- Scoreboard checks (expected byte from sequence vs observed byte on MOSI).
- Final scoreboard summary (matches and mismatches).

Demo: SPI UVM

```
$ ./scripts/module6.sh --run
```

Terminal: spi_uvm

```
[0:36m=====0m
[0:36mModule 6: Run SPI UVM example (logging to /mnt/d/proj/universal-verification-methodology/lear
[0:36m=====0m

[1:33m[INFO][0m Setting UVM_HOME to: /mnt/d/proj/universal-verification-methodology/learn_uart_spi
[Thu May 28 15:27:16 CST 2026] Running make SIM=verilator TEST=test_spi_uvm
Compiling SPI UVM test with Verilator...
verilator -sv --timing --trace -Wno-fatal -Wno-TIMESCALEMOD -Wno-WIDTHTRUNC -I/mnt/d/proj/universal-
--binary \
/mnt/d/proj/universal-verification-methodology/learn_uart_spi_i2c/tools/uvm-2017/1800.2-2017-1.0/sr
dut/*.v test_spi_uvm.sv \
--top-module test_spi_uvm
/bin/sh: 1: verilator: not found
make: *** [Makefile:40: compile] Error 127
```

Practice & assessment

What you should know

- Describe the SPI UVM agent (transaction, sequence, driver, monitor, scoreboard) and mode 0 monitoring.
- Run `spi_uvm` and interpret UVM output.
- Ready for Module 7: I²C protocol + RTL + basic testbench.

Exercises

- Run spi_uvm
- Trace one transaction
- Change the sequence
- Optional: Compare to UART UVM

Assessment checklist

- Can describe the SPI UVM agent and how it maps to SPI (one byte, mode 0 monitoring).
- Can run `spi_uvm` and interpret UVM output.
- Ready for Module 7: I²C protocol + RTL + basic testbench.

Summary & next steps

- Extend SPI verification to ****UVM+SV**** — SPI agent
(transaction, sequence, driver, monitor, scoreboa...
- Complete module6/CHECKLIST.md
- Review module6/EXAMPLES.md and run each lab
- Next: I²C